

Hurghiș Gheorghe-Georgian  
Guriuc Vald-Ionuț  
Grupa 1405B

# **Predicția calității vinului roșu**

Disciplina: Inteligența artificială

Profesor îndrumător: Mircea Hulea

Echipa:

Hurghiș Gheorghe-Georgian  
Guriuc Vlad-Ionuț

### Cerinta proiect:

Antrenarea unei rețele neuronale de tip perceptron multistrat cu o structură predefinită (un număr specificat de straturi ascunse și neuroni) cu ajutorul unui algoritm evolutiv (tema cu numărul 10).

### Indicatii:

Mai întâi, se va alege o problemă de clasificare sau de regresie pe care să o învețe rețeaua neuronală. Se poate alege o problemă din UCI Repository: <https://archive.ics.uci.edu/ml/index.php>. Structura rețelei se consideră stabilită apriori, la fel și funcția de activare. Se recomandă 1-2 straturi ascunse și un număr de neuroni în fiecare strat potrivit pentru problema aleasă. Algoritm evolutiv va fi utilizat pentru determinarea ponderilor conexiunilor și pragurilor neuronilor din rețea. În final, se vor afișa rezultatele obținute de rețea pentru mulțimea de antrenare a problemei considerate.

### Descrierea problemei considerate

Am abordat tema “Predicția calității vinului roșu” pentru tema primită preponderent datorită dataset-ului (<https://archive.ics.uci.edu/dataset/186/wine+quality>). Dataset-ul nu are valori lipsă și abordează atât problema clasificării, cât și problema regresiei. Pe de altă parte, în pagina site-ului corespunzător dataset-ului ales, la rubrica informații suplimentare, se precizează incertitudinea utilității anumitor parametri. Site-ul atenționează că unele valori ar putea fi (sau deveni) irelevante (sau înșelătoare) în anumite circumstanțe din soluțiile abordate, fapt pentru care randamentul de detecție ar putea avea un indice mai scăzut decât cel așteptat.

### Aspecte teoretice privind algoritmul și mod de rezolvare

În tabelul de mai jos regăsim parametrii luați în considerare în setul de date și utilizați ulterior ca intrări în algoritm.

| Variables Table      |         |            |
|----------------------|---------|------------|
| Variable Name        | Role    | Type       |
| fixed_acidity        | Feature | Continuous |
| volatile_acidity     | Feature | Continuous |
| citric_acid          | Feature | Continuous |
| residual_sugar       | Feature | Continuous |
| chlorides            | Feature | Continuous |
| free_sulfur_dioxide  | Feature | Continuous |
| total_sulfur_dioxide | Feature | Continuous |
| density              | Feature | Continuous |
| pH                   | Feature | Continuous |
| sulphates            | Feature | Continuous |

Hurghiș Gheorghe-Georgian  
Guriuc Vald-Ionuț  
Grupa 1405B

Dupa cum am precizat anterior, este posibil ca o parte din aceste valori sa fie inselatoare in anumite circumstante. Am implementat in algoritm o serie de functii specifice:

- pentru activare, derivare:

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))  
  
def sigmoid_derivative(x):  
    return x * (1 - x)
```

1. sigmoid(x); este folosita pentru a introduce non-linearitati in retelele neuronale.
2. sigmoid\_derivative(x); pentru propagare de erori in timpul antrenarii.

- pentru normalizare si eroare:

```
def softmax(x):  
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))  
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)  
  
# calcul a erorii cross-entropy  
def cross_entropy_loss(y_true, y_predicted):  
    return -np.sum(y_true * np.log(y_predicted + 1e-9)) / y_true.shape[0]
```

1. softmax(x); transforma valorile din vectorul x in probabilitati si normalizeaza valorile exponentiale astfel incat suma probabilitatilor sa fie 1.
2. cross\_entropy\_loss(y\_true, y\_predicted); calculeaza eroarea intre valorile adevarate si cele prezise de algoritm. Valoarea 1e-9 este folosita pentru a evita log(0);

- pentru initializare si fitness:

```
def initialize_population(population_size, num_weights):  
    return [np.random.uniform(-1, 1, num_weights) for _ in range(population_size)]  
  
# evaluare fitness pentru un individ  
def fitness_eval(individ, X, y, structure):  
    weights, biases = decode_individ(individ, structure)  
    predictions = forward_pass(X, weights, biases)  
    loss = cross_entropy_loss(y, predictions)  
    return -loss # fitness-ul este negativul erorii
```

1. initialize\_population(population\_size, num\_weights); genereaza o populatie initiala de indivizi, fiecare fiind un vector de greutati(w) aleatoare intre -1 si 1.
2. fitness\_eval(individ, X, y, structure); evalueaza un individ/ o solutie calculand fitness-ul sau(decodeaza in greutati si bias, realizeaza trecerea in retea, calculeaza eroarea si returneaza negativul erorii pentru a minimiza eroarea.

- pentru decodificarea si trecerea prin retea:

Hurghiș Gheorghe-Georgian

Guriuc Vald-Ionuț

Grupa 1405B

```
# decodificare individ
def decode_individ(individ, structure):
    weights = []
    biases = []
    index = 0
    for i in range(len(structure) - 1):
        intrare_dim = structure[i]
        iesire_dim = structure[i + 1]
        weight_count = intrare_dim * iesire_dim
        bias_count = iesire_dim

        weights.append(individ[index:index + weight_count].reshape(intrare_dim, iesire_dim))
        index += weight_count
        biases.append(individ[index:index + bias_count])
        index += bias_count
    return weights, biases

# trecere prin retea
def forward_pass(X, weights, biases):
    layer_intrare = X

    for i in range(len(weights) - 1):
        layer_iesire = sigmoid(np.dot(layer_intrare, weights[i]) + biases[i])
        layer_intrare = layer_iesire

    final_output = softmax(np.dot(layer_intrare, weights[-1]) + biases[-1])
    return final_output
```

1. `decode_individ(individ, structure)`; transforma individul intr-un set de greutate si bias-uri.
  2. `forward_pass(X, weights, biases)`; realizeaza trecerea prin retea cu ajutorul functiilor sigmoid pentru straturile ascunse si stratului de iesire i se aplica `softmax(x)`.
- pentru selectie, incrucisare si mutatie:

Hurghiș Gheorghe-Georgian  
Guriuc Vald-Ionuț  
Grupa 1405B

```
# selectia parintilor pe baza fitnessului in turneu
def select_parents(population, fitness, tournament_size=3):
    parents = []
    for _ in range(2):
        tournament = random.sample(range(len(population)), tournament_size)
        best = max(tournament, key=lambda i: fitness[i])

        parents.append(population[best])
    return parents

# incrucisare intre 2 parinti
def crossover(parinte1, parinte2, crossover_rate=0.7):
    if random.random() < crossover_rate:
        point = random.randint(1, len(parinte1) - 1)

        copil1 = np.concatenate((parinte1[:point], parinte2[point:]))
        copil2 = np.concatenate((parinte2[:point], parinte1[point:]))
        return copil1, copil2
    return parinte1.copy(), parinte2.copy()

# mutarea unui individ
def mutate(individual, mutation_rate=0.1):
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            individual[i] += np.random.normal(0, 0.1)
    return individual
```

1. `select_parents(population, fitness, tournament_size=3)`; selecteaza indivizii cu cel mai bun fitness.
  2. `crossover(parinte1, parinte2, crossover_rate=0.7)`; creeaza un nou individ combinand partile parintilor.
  3. `mutate(individual, mutation_rate=0.1)`; pentru fiecare gena adauga un zgomot.
- functia principala de antrenare:

Hurghiș Gheorghe-Georgian

Guriuc Vald-Ionuț

Grupa 1405B

```
def train_evolutionary(X, y, structure, population_size=50, generations=50, mutation_rate=0.2):
    nr_weights = sum(structure[i] * structure[i + 1] + structure[i + 1] for i in range(len(structure) - 1))

    population = initialize_population(population_size, nr_weights)

    for generation in range(generations):
        fitness = [fitness_eval(individ, X, y, structure) for individ in population]
        new_population = []

        for _ in range(population_size // 2):
            parinte1, parinte2 = select_parents(population, fitness)
            copil1, copil2 = crossover(parinte1, parinte2)

            new_population.append(mutate(copil1, mutation_rate))
            new_population.append(mutate(copil2, mutation_rate))

        population = new_population
        best_fitness = max(fitness)

        print(f"generatia {generation + 1}: cel mai bun fitness = {best_fitness}")

    best_individ = population[np.argmax(fitness)]
    weights, biases = decode_individ(best_individ, structure)
    return weights, biases
```

1. train\_evolutionary(); initializeaza populatia, evalueaza fitness-ul indivizilor, selecteaza iterativ parinti pentru a crea o noua generatie si va returna cel mai bun individ.

- pentru evaluarea performantei:

```
def evaluate_performance(predictions, labels):
    predicted_classes = np.argmax(predictions, axis=1) + 1
    correct = np.sum(predicted_classes == labels)

    accuracy = correct / len(labels)
    print(f"acuratetea retelei: {accuracy * 100:.2f}%")
    return accuracy
```

1. evaluate\_performance(predictions, labels) ; calculeaza si afiseaza randamentul algoritmului in epoca actuala comparand predictiile proprii cu etichetele adevarate.

Hurghiș Gheorghe-Georgian

Guriuc Vald-Ionuț

Grupa 1405B

## Rezultatele obținute prin rularea programului în diverse situații, capturi ecran și comentarii asupra rezultatelor obținute

Exemplu integral rulare epoca ce are structura [11,6,7,10]

```
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]
generatia 1: cel mai bun fitness = -1.6663989614768955
generatia 2: cel mai bun fitness = -1.5658542422442424
generatia 3: cel mai bun fitness = -1.4701918832002013
generatia 4: cel mai bun fitness = -1.338928080699277
generatia 5: cel mai bun fitness = -1.3561286801115868
generatia 6: cel mai bun fitness = -1.3051913536066986
generatia 7: cel mai bun fitness = -1.2904899356671615
generatia 8: cel mai bun fitness = -1.2770188201356254
generatia 9: cel mai bun fitness = -1.2573348280586087
generatia 10: cel mai bun fitness = -1.2525798601083817
generatia 11: cel mai bun fitness = -1.2472885591059133
generatia 12: cel mai bun fitness = -1.2346915541324586
generatia 13: cel mai bun fitness = -1.221760841310529
generatia 14: cel mai bun fitness = -1.2204142202891919
generatia 15: cel mai bun fitness = -1.212434121862342
generatia 16: cel mai bun fitness = -1.2165749610472045
generatia 17: cel mai bun fitness = -1.2208381489997455
generatia 18: cel mai bun fitness = -1.2158652343207748
generatia 19: cel mai bun fitness = -1.2191286457273858
generatia 20: cel mai bun fitness = -1.2151913096774956
generatia 21: cel mai bun fitness = -1.2035088719952853
generatia 22: cel mai bun fitness = -1.19150880473365443
generatia 23: cel mai bun fitness = -1.1914038457047182
generatia 24: cel mai bun fitness = -1.2020390583021814
generatia 25: cel mai bun fitness = -1.1895642899953187
generatia 26: cel mai bun fitness = -1.1894620805767218
generatia 27: cel mai bun fitness = -1.1866880863357807
generatia 28: cel mai bun fitness = -1.1854375044273817
generatia 29: cel mai bun fitness = -1.1813958718047077
generatia 30: cel mai bun fitness = -1.1800486018778944
generatia 31: cel mai bun fitness = -1.1826960767590682
generatia 32: cel mai bun fitness = -1.1800184072521047
generatia 33: cel mai bun fitness = -1.1758101670629004
generatia 34: cel mai bun fitness = -1.175072879173558
generatia 35: cel mai bun fitness = -1.1738689510200265
generatia 36: cel mai bun fitness = -1.1735432113105213
generatia 37: cel mai bun fitness = -1.1711811505116165
generatia 38: cel mai bun fitness = -1.172763936390071
generatia 39: cel mai bun fitness = -1.1713601375961577
generatia 40: cel mai bun fitness = -1.1681297376237203
generatia 41: cel mai bun fitness = -1.1647402360893118
generatia 42: cel mai bun fitness = -1.163109613780219
generatia 43: cel mai bun fitness = -1.1622961598536106
generatia 44: cel mai bun fitness = -1.1626219351284393
generatia 45: cel mai bun fitness = -1.1580931473841487
generatia 46: cel mai bun fitness = -1.1571214351056494
generatia 47: cel mai bun fitness = -1.154896048664586
generatia 48: cel mai bun fitness = -1.1554660685284937
generatia 49: cel mai bun fitness = -1.1529435669954802
generatia 50: cel mai bun fitness = -1.150782159377507
predicții obținute: [5 5 ... 5 5 5]
acuratetea rețelei: 42.59%
: 0.425891181988743
```

Exemplu integral rulare epoca ce are structura [11,3,4,10]

# Hurghiș Gheorghe-Georgian

## Guriuc Vald-Ionuț

### Grupa 1405B

```
generatia 1: cel mai bun fitness = -1.7203734326524789
generatia 2: cel mai bun fitness = -1.6099436522334867
generatia 3: cel mai bun fitness = -1.370320190204352
generatia 4: cel mai bun fitness = -1.374880096004563
generatia 5: cel mai bun fitness = -1.3942324167438154
generatia 6: cel mai bun fitness = -1.2962143568714204
generatia 7: cel mai bun fitness = -1.3511904427663668
generatia 8: cel mai bun fitness = -1.3072148601214595
generatia 9: cel mai bun fitness = -1.2821629565110553
generatia 10: cel mai bun fitness = -1.2674505210009772
generatia 11: cel mai bun fitness = -1.2675318353845046
generatia 12: cel mai bun fitness = -1.251948101934115
generatia 13: cel mai bun fitness = -1.2432998952068723
generatia 14: cel mai bun fitness = -1.235571885827149
generatia 15: cel mai bun fitness = -1.2276827332905762
generatia 16: cel mai bun fitness = -1.2222109217082935
generatia 17: cel mai bun fitness = -1.214866422099721
generatia 18: cel mai bun fitness = -1.2103514087014036
generatia 19: cel mai bun fitness = -1.2073888344419852
generatia 20: cel mai bun fitness = -1.202266750343723
generatia 21: cel mai bun fitness = -1.207709025154558
generatia 22: cel mai bun fitness = -1.2009230816066558
generatia 23: cel mai bun fitness = -1.1969233499105345
generatia 24: cel mai bun fitness = -1.1930660043424821
generatia 25: cel mai bun fitness = -1.1917457289862456
generatia 26: cel mai bun fitness = -1.1909447099899417
generatia 27: cel mai bun fitness = -1.1835902724970058
generatia 28: cel mai bun fitness = -1.1833736783890945
generatia 29: cel mai bun fitness = -1.1863709185148161
generatia 30: cel mai bun fitness = -1.1781008197774216
predictii obtinute: [6 6 6 ... 6 6 6]
acuratetea retelei: 38.71%
```

### Exemplu partial rulare epoca avand structura [11,7,10]

```
generatia 47: cel mai bun fitness = -1.1423727411867923
generatia 48: cel mai bun fitness = -1.140738555843469
generatia 49: cel mai bun fitness = -1.1336469031465901
generatia 50: cel mai bun fitness = -1.1401663068854597
predictii obtinute: [6 6 6 ... 6 6 6]
acuratetea retelei: 46.40%
```

### Exemplu partial rulare epoca avand structura [11,5,5,10]

```
generatia 397: cel mai bun fitness = -1.0766608005438147
generatia 398: cel mai bun fitness = -1.0764325475003211
generatia 399: cel mai bun fitness = -1.0568539951507983
generatia 400: cel mai bun fitness = -1.0634083016068245
predictii obtinute: [5 6 6 ... 6 6 6]
acuratetea retelei: 42.84%
```

[44]:

0.4283927454659162

## Concluzii

Antrenarea rețelei neuronale de tip perceptron multistrat cu o structură predefinită cu ajutorul unui algoritm evolutiv nu atinge pragul optim de detectie, iar marja de acuratete a rețelei s-a situat(din exemple ilustrate si neilustrate) de la 37% la 52%.

## Bibliografie

- <https://archive.ics.uci.edu>
- Laboratoare+Cursuri IA moodle
- GeeksforGeeks



Hurghiș Gheorghe-Georgian  
Guriuc Vald-Ionuț  
Grupa 1405B

### **Listă contributii**

Hurghiș Gheorghe-Georgian- proiectare, dezvoltare, integrare, depanare algoritm retea neuronală, rulare a unor epoci;

Guriuc Vlad-Ionuț- testare epoci, depanare algoritm retea neuronală, documentatie